CS612 · ADVANCED ALGORITHMS · MSC · FALL 2026

Lecture 03
Randomized Algorithms & Probabilistic Analysis

Randomness as a design tool: what exactly is guaranteed, over what, and against whom?

Saturday 10 October 2026 · 3 hours

Project specification released today · PS1 due Sat 17 Oct

Where we are

L02 left two loose ends

- Randomized quickselect: "expected $O(n)$", with an intuitive argument. Where exactly does $3.4n$ come from?
- "Robust to adversarial input *if* the adversary cannot predict the pivots". What does that mean formally?

TODAY

A precise vocabulary for randomized guarantees (**Las Vegas / Monte Carlo**, expected time, failure probability, adversary model) and the core technique: **indicator random variables + linearity of expectation**.

Two motivating problems

1 · The killer adversary

McIlroy (1999) wrote a short program that, *while* a library's quicksort runs, answers its comparisons so as to force $\Theta(n^2)$ behaviour for essentially any deterministic pivot rule, including median-of-3.

Real systems have been attacked with crafted inputs (sorting, hashing).

Can any algorithm be safe against this?

2 · Trust but verify

A GPU service returns C and claims $C = AB$ for $n \times n$ matrices. Recomputing AB costs $\Theta(n^3)$ (or $n^{2.8}$).

Can we check the claim in $O(n^2)$ time, if we accept a tiny chance of being fooled?

DISCUSS · 3 MIN

For each: what would "safe" or "checked" mean *precisely*? What are we willing to give up?

BLOCK A · 00:10–01:00

What does a randomized algorithm guarantee?

Las Vegas vs. Monte Carlo · expected time · adversaries · amplification

Why randomize?

Reason	Mechanism	Examples in CS612
Defeat the adversary	No fixed input is bad for all random choices	randomized quicksort/quickselect (today), universal hashing (L04)
Fingerprinting	Compare short random "sketches" instead of large objects	Freivalds (today), Rabin–Karp (L11)
Simplicity	Random structure replaces complex balancing rules	skip lists vs. red-black trees (L04), treaps
Sampling	A small random sample represents the whole	percentile estimation, reservoir sampling (exercise)
Symmetry breaking	Random choices resolve ties/conflicts without coordination	distributed algorithms (not covered)

> Randomization is not "hoping the input is random". The **algorithm** flips the coins; the input can be the worst possible.

Las Vegas vs. Monte Carlo

	Las Vegas	Monte Carlo
Answer	**always correct**	correct with probability $\geq 1 - \delta$
Running time	random: we bound $\mathbb{E}[T]$ (and its tail)	bounded (usually deterministic)
Examples	randomized quicksort, quickselect; Rabin–Karp *with* verification; skip-list search	Freivalds; Miller–Rabin primality; Rabin–Karp *without* verification
Error types	none (only slowness)	**one-sided** (e.g. never rejects a correct C) or **two-sided**

LV → MC

Stop after $2\,\mathbb{E}[T]$ steps and output "fail". By Markov, $\Pr[T \geq 2\mathbb{E}[T]] \leq \frac{1}{2}$.

MC → LV

Possible if answers can be **verified**: repeat until a verified answer appears. Expected trials $= 1/p$.

"Expected time" means something precise

$$T_{\exp}(n) = \max_{|x|=n} \mathbb{E}_r\left[T(x, r)\right]$$

- The **max over inputs** is outside: a worst-case input, chosen first.
- The **expectation over our coins** is inside.
- Contrast with average case: $\mathbb{E}_{x \sim \mathcal{D}}[T(x)]$, which trusts a model of the inputs.

Adversary models

- **Oblivious:** fixes the input (or operation sequence) before any coin flip. Our guarantees hold.
- **Adaptive:** sees our random choices (or can predict them) and reacts. Expected-time guarantees can fail.

WHERE ADAPTIVE ADVERSARIES APPEAR

- A predictable random generator: fixed seed, time-based seed
- An online data structure whose behaviour (timing!) leaks the random choices
- Hash functions with a public seed: hash flooding (L04)

Engineering rule: seed from a good entropy source **once**, keep the seed secret where attackers exist, and **log it** for reproducibility.

Freivalds' algorithm (1977)

```
1  FREIVALDS(A, B, C, k)        ▷ n × n matrices
2    repeat k times
3      choose r ∈ {0,1}^n uniformly at random
4      if A·(B·r) ≠ C·r          ▷ three matrix–vector products
5        return "C ≠ AB"         ▷ certainly correct
6    return "C = AB"             ▷ possibly wrong
```

Cost: $O(kn^2)$ instead of $\Theta(n^3)$.

THEOREM

If $C = AB$, the algorithm always accepts. If $C \neq AB$, one trial rejects with probability $\geq \frac{1}{2}$, so the algorithm wrongly accepts with probability $\leq 2^{-k}$.

Monte Carlo with **one-sided error**.

Proof: the principle of deferred decisions

PROOF

Let $D = AB - C \neq 0$, so some entry $d_{ij} \neq 0$. A trial accepts iff $Dr = 0$, which implies $(Dr)_i = 0$:

$$(Dr)_i = \sum_{\ell} d_{i\ell} r_\ell = d_{ij} r_j + y, \qquad y = \sum_{\ell \neq j} d_{i\ell} r_\ell .$$

Condition on all coordinates except r_j, which fixes y. Then $(Dr)_i = 0$ iff $r_j = -y/d_{ij}$. At most one of the two values $r_j \in \{0, 1\}$ satisfies this, so

$$\Pr[Dr = 0] \leq \Pr[(Dr)_i = 0] \leq \tfrac{1}{2}.$$

The k trials are independent, so $\Pr[\text{all accept}] \leq 2^{-k}$.

■

TECHNIQUE

Deferred decisions: fix all random choices except one, then show that the bad event constrains that last choice.

Amplification: how many repetitions?

One-sided error p

Repeat k times, and accept only if all runs accept: error $\leq p^k$.

For δ: $k = \lceil \log(1/\delta) / \log(1/p) \rceil$. Freivalds: $\delta = 10^{-9}$ needs $k = 30$.

Two-sided error $\leq \frac{1}{2} - \varepsilon$

Repeat k times and take the **majority**. Chernoff–Hoeffding:

$$\Pr[\text{majority wrong}] \leq e^{-2\varepsilon^2 k}.$$

So $k = O(\varepsilon^{-2} \log(1/\delta))$.

COST OF CERTAINTY

Each factor of 10 in confidence costs a *constant* number of extra trials: $\log_2 10 \approx 3.3$ for Freivalds.

INDEPENDENCE IS ESSENTIAL

Re-running with the **same** random bits (e.g. re-seeding with the same time value) gives the same error again, and amplification does nothing.

Exercise A: classify the guarantee

Algorithm	LV or MC?	One/two-sided?	What is random, and what is bounded?
Randomized quicksort			
Freivalds			
Miller–Rabin primality test			
Rabin–Karp, report on hash match			
Rabin–Karp, verify each hash match			
Estimate the median from a random sample of 1000 values			

Pairs · 6 min

BLOCK B · 01:10–02:00

Analysing randomized quicksort and quickselect

Indicators · linearity of expectation · exact constants · concentration

Randomized quicksort: what to count

```
1 RQS(A, lo, hi)
2  if lo < hi
3     p = uniform random index in [lo, hi]
4     swap A[p], A[hi]
5     q = PARTITION(A, lo, hi)   ▷ Lomuto
6     RQS(A, lo, q − 1)
7     RQS(A, q + 1, hi)
```

Let the keys in sorted order be $z_1 < z_2 < \cdots < z_n$ (distinct).

- Running time $= \Theta(n + X)$, where X = the total number of key comparisons.
- Every comparison is between the **current pivot** and another element of its subarray.
- So each pair z_i, z_j is compared **at most once**: after partitioning, the pivot is never used again.

$$X = \sum_{i<j} X_{ij}, \qquad X_{ij} = 1[z_i \text{ and } z_j \text{ are compared}]$$

The key lemma

LEMMA

$$\Pr[z_i \text{ and } z_j \text{ are compared}] = \frac{2}{j - i + 1} \text{ for } i < j.$$

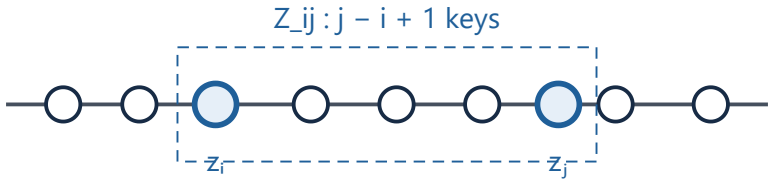
PROOF

Let $Z_{ij} = \{z_i, z_{i+1}, \ldots, z_j\}$, so $|Z_{ij}| = j - i + 1$. Until some element of Z_{ij} becomes a pivot, all of Z_{ij} stay in the same subarray, because any pivot outside Z_{ij} is smaller or larger than all of them. Consider the **first** pivot chosen from Z_{ij}:

- If it is z_i or z_j: the two are compared (the pivot meets every element of its subarray).
- If it is some z_ℓ with $i < \ell < j$: z_i and z_j are separated forever and never compared.

Each element of Z_{ij} is equally likely to be that first pivot (pivots are uniform within the current subarray, which contains all of Z_{ij}). So the probability is $2/(j - i + 1)$.

■



first pivot inside the box decides:
an endpoint → compared · interior → never

Summing: $\mathbb{E}[X] \approx 1.39\, n \log_2 n$

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{d=2}^{n-i+1} \frac{2}{d} \;<\; 2n\, H_n = 2n \ln n + O(n).$$

Exact value: $\mathbb{E}[X] = 2(n+1)H_n - 4n$. Since $2 \ln n = (2 \ln 2) \log_2 n \approx 1.386 \log_2 n$:

	comparisons
Lower bound $\lceil \log_2 n! \rceil$ (L01)	$n \log_2 n - 1.44n$
Merge sort (worst case)	$\approx n \log_2 n - n$
Randomized quicksort (expected)	$\approx 1.39\, n \log_2 n - 2.85n$

> Quicksort makes about **39% more comparisons** than optimal in expectation. It usually wins anyway: it is in place, has sequential access and a tight inner loop.

Beyond expectation: concentration

An expectation alone does not rule out frequent slow runs. For quicksort, much more is true:

FACT (HIGH PROBABILITY)

For any constant c, there is a c' such that randomized quicksort makes at most $c' n \ln n$ comparisons with probability $\geq 1 - n^{-c}$.

Sketch. Follow one element. At each recursion level its subarray shrinks by a factor $\leq 3/4$ with probability $\geq 1/2$ (the pivot lands in the middle half). After $\log_{4/3} n$ such "good" levels the subarray has size 1. A Chernoff bound shows that more than $8 \log_{4/3} n$ levels occur with probability $\leq n^{-2}$. A union bound over n elements gives a recursion depth of $O(\log n)$ with probability $\geq 1 - 1/n$. Each level costs $\leq n$.

Spread: the standard deviation of X is $\sim 0.65\, n$ (Knuth), while the mean is $\sim 2n \ln n$. Relative spread $\to 0$ as n grows.

WHY ENGINEERS CARE

Latency SLOs are about **tails** (p99), not means. A guarantee "with probability $1 - n^{-c}$" says something about the tail. An expectation alone says nothing.

Randomized quickselect: expected $\leq 4n$ comparisons

Selecting rank k. Pair z_i, z_j ($i < j$) is compared iff the first pivot chosen from $\{z_{\min(i,k)}, \ldots, z_{\max(j,k)}\}$ is z_i or z_j:

$$\Pr[z_i, z_j \text{ compared}] = \frac{2}{\max(j,k) - \min(i,k) + 1}.$$

Split the pairs into three groups:

- $i < j \leq k$: probability $\frac{2}{k-i+1}$, and $k - i$ such j per i → total $\leq 2k$.
- $k \leq i < j$: symmetric → total $\leq 2(n - k + 1)$.
- $i < k < j$: for span $d = j - i + 1$ there are $\leq d - 2$ such pairs → $\sum_d \frac{2(d-2)}{d} \leq 2n$.

$\mathbb{E}[\text{comparisons}] \leq 4n + O(1)$: linear in expectation.

Exact for the median: $2(1 + \ln 2)\, n \approx \mathbf{3.386\,n}$.
Measured in L02: 3.4n.

Implementation: getting the randomness right

Legacy code (two bugs)

```
int randomizedPartition(int A[], int low, int high) {
    srand(time(NULL));                          // BUG 1
    int randomIndex = low + rand() % (high - low); // BUG 2
    swap(A[randomIndex], A[high]);
    return partition(A, low, high);
}
```

Correct

```
static std::mt19937_64 rng(seed);  // seeded ONCE; seed logged
size_t pick(size_t lo, size_t hi) {
    std::uniform_int_distribution<size_t> d(lo, hi); // inclusive
    return d(rng);
}
```

ASK

1. What does bug 1 do to the "random" pivots within one second?

2. Which element can bug 2 never choose? Does that break the $2/(j-i+1)$ lemma?

3. Is `rand() % m` uniform?

BLOCK C · 02:10–02:55

Choosing probabilistic guarantees

Questions to ask · an experiment · design exercise

Five questions before trusting a randomized algorithm

1. **What can go wrong: time or answer?** (LV vs. MC.) Which failure is acceptable in this application?
2. **Over what is the probability taken?** Our coins only? Or also an assumption about inputs?
3. **Who is the adversary?** Oblivious (the guarantee holds) or adaptive / able to predict our randomness (it may not)?
4. **Can answers be verified?** If so, MC can become LV, and errors can be caught downstream.
5. **What does amplification cost?** Repetitions to reach the target δ, and are the repetitions really independent?

> And an engineering sixth: **reproducibility**. Log seeds; make experiments rerunnable; separate "secret seed in production" from "fixed seed in testing".

Experiment: Freivalds in practice

$n = 600$, 64-bit integers	time
Classical multiplication $\Theta(n^3)$	114.0 ms
Freivalds, $k = 30$ trials (error $\leq 2^{-30}$)	**15.6 ms**

Single-trial detection rate (2000 trials)	measured	theory
C has **one** wrong entry	0.490	0.5
One wrong entry in every row (distinct columns)	1.000	$1 - 2^{-600}$

`Student_Resources/freivalds.cpp`, g++ 15.2 -O2.

DISCUSS · 4 MIN

1. How does the time ratio change at $n = 6000$?

2. The single-entry case sits exactly at the bound. Why is the bound tight there?

3. What does this experiment **not** tell you about the error probability?

Exercise C: design with randomness

1. **Budget.** For $n = 10^4$, how many Freivalds trials give error $\le 10^{-12}$? Compare the cost in scalar multiplications with recomputing AB.
2. **LV → MC.** Randomized quickselect uses $\le 4n$ comparisons in expectation. Give a Monte Carlo version that *always* stops within $40n$ comparisons, and bound its failure probability. Can you make the failure probability 2^{-10} within $100n$?
3. **Reservoir sampling ($k = 1$).** Read a stream $x_1, x_2, \ldots$ of unknown length. Keep $s = x_1$, and for $t \ge 2$ replace s by x_t with probability $1/t$. Prove that after n items, $\Pr[s = x_i] = 1/n$ for every i.
4. **Adversary.** A web service sorts user-supplied arrays with randomized quicksort, seeded with the server start time, and the response time is visible. Describe an attack, and a fix.

Groups · 15 min

Synthesis

1. **Las Vegas** (always correct, random time) vs. **Monte Carlo** (bounded time, error $\leq \delta$, one- or two-sided).
2. Expected time = $\max_x \mathbb{E}_r[T(x, r)]$: worst-case input, our coins. Valid against **oblivious** adversaries only.
3. **Indicators + linearity**: decompose a complicated cost into 0/1 events and add their probabilities. Quicksort: $2(n+1)H_n - 4n$; quickselect: $\leq 4n$.
4. **Deferred decisions** and **amplification**: error 2^{-k} costs k independent runs.
5. Tails matter: high-probability bounds, not just expectations.

NEXT WEEK · L04 (SAT 17 OCT)

Hashing and randomized data structures: universal hashing (a random function beats the adversary), perfect hashing (worst-case $O(1)$ lookups), skip lists (expected $O(\log n)$, *not* worst case).

Read: CLRS 4e Ch. 11; Pugh 1990. **Due 17 Oct:** PS1 and project M1 (team + topic).